

APRENDIZAJE DE MÁQUINAS (ACIF104) INFORME FASE 3 — SUMATIVA 2

Nombre integrante/s:

Javiera Chávez
Joaquín Olivares
Claudio Yáñez

Curso: Aprendizaje de Máquina

Fecha: 18-08-2026

Índice

- Índice2
- 1. Introducción3
 - 1.1. Respuesta a la retroalimentación de la Sumativa 13
- 2. Problemática4
 - 2.1. Requisitos iniciales.....4
- 3. Metodología.....5
 - 3.1. Fases aplicadas hasta ahora5
 - 3.2. Plan de trabajo detallado5
- 4. Desarrollo del proyecto6
 - 4.1. Comparación de técnicas de Machine Learning y arquitecturas de Deep Learning.....6
 - 4.2. Requisitos funcionales y no funcionales7
 - 4.3. Arquitectura: justificación de la selección del modelo8
 - 4.4. Modelos: partición de datos y balanceo de clases9
 - 4.5. Sistema: avance en la integración frontend/backend 10
- 5. Resultados 11
 - 5.1. Análisis interpretativo mediante SHAP..... 12
- 6. Propuesta de mejoras..... 13
 - 6.1. Limitaciones identificadas 13
 - 6.2. Mejoras propuestas..... 13
- 7. Código fuente en GitHub 14
- 8. Referencias..... 14

1. Introducción

Este informe corresponde a la tercera fase del proyecto Perfectime, desarrollado en el marco de la asignatura Aprendizaje de Máquina (ACIF104). El trabajo es de naturaleza iterativa e incremental: no parte de cero, sino que toma la problemática, la metodología (CRISP-DM) y el análisis del informe anterior, y aplica sobre ellos las correcciones y mejoras señaladas en la retroalimentación de la Sumativa 1, además de profundizar en la validación y optimización de los modelos predictivos.

El documento se organiza en las siguientes secciones: respuesta a la retroalimentación recibida, problemática y requisitos iniciales, metodología y plan de trabajo, desarrollo del proyecto (comparación de técnicas, requisitos, arquitectura del modelo, partición y balanceo de datos, y avance del sistema), resultados y su interpretación mediante SHAP, propuesta de mejoras, código fuente en GitHub y referencias en formato APA 7.^a edición.

1.1. Respuesta a la retroalimentación de la Sumativa 1

La siguiente tabla resume, punto por punto, las observaciones recibidas del docente sobre la entrega anterior y la acción concreta tomada en esta fase para resolverlas.

Observación recibida	Acción tomada en esta fase	Evidencia
Repositorio no enlazado en el informe; README insuficiente (no explica instalación ni environment.yml)	Se incorporó el enlace explícito al repositorio y se actualizó el README con instrucciones detalladas de creación del entorno Conda desde environment.yml y de ejecución del notebook, backend y frontend	Sección 7; README.md
El sistema (frontend/backend) no era funcional: solo una maqueta visual sin conexión al modelo	Se construyó desde cero una API REST con FastAPI que carga el modelo XGBoost serializado y lo sirve en tiempo real; se conectó la pantalla "Predicción individual" del frontend al backend mediante fetch, con validación de errores	Sección 4.e; backend/main.py
Cálculo erróneo de AUC-ROC (0,8785) usando etiquetas predichas en vez de probabilidades; el valor correcto (0,978) aparecía más adelante pero no se usó para las decisiones	Se corrigió el cálculo en la función evaluar_modelo() y en toda la tabla comparativa, usando predict_proba() de forma consistente en los seis modelos	Sección 4.a, Tabla 3
Afirmaciones sin respaldo en el código: cálculo de "PR-AUC" para 6 técnicas que incluían SVM, 1D-CNN y LSTM (nunca entrenadas), y uso de scale_pos_weight (no implementado)	Este informe reporta únicamente las técnicas efectivamente entrenadas y verificadas en el notebook (3 de ML y 3 de DL, todas ejecutadas de extremo a extremo), sin afirmaciones no respaldadas por código	Sección 4.a
Faltaba comparar sin balanceo / submuestreo / SMOTE con las mismas métricas, y ajustar hiperparámetros con GridSearchCV	Se implementaron y ejecutaron los tres escenarios de balanceo bajo las mismas métricas, y se realizó la búsqueda de hiperparámetros con GridSearchCV sobre XGBoost (108 combinaciones × 5 folds)	Sección 4.d, Tabla 7; sección 4.c
Requisitos no conectados con decisiones de preprocesamiento; faltaba documentar que la eliminación de columnas por fuga de datos resuelve RF-05	Se documentó explícitamente, junto a la tabla de requisitos funcionales, la eliminación de TWF/HDF/PWF/OSF/RNF por fuga de datos y su relación directa con RF-05	Sección 4.b

Observación recibida	Acción tomada en esta fase	Evidencia
Errores de formato APA 7: título "Biografía" en vez de "Referencias", autores incompletos ("et al."), faltaban volumen/número/páginas/DOI, revista incorrecta de Benhanifia et al., y referencias faltantes de Breiman y Cortes & Vapnik	Se renombró la sección a "Referencias", se completaron todos los autores, volumen, número, páginas y DOI, se corrigió la revista de Benhanifia et al. a Intelligent Systems with Applications, y se agregaron las referencias faltantes	Sección 8

Tabla 1. Respuesta a la retroalimentación de la Sumativa 1.

2. Problemática

La transformación digital impulsada por la Industria 4.0 ha incrementado de manera significativa la cantidad de información disponible para las organizaciones. Los procesos industriales actuales incorporan sensores, dispositivos IoT y sistemas de monitoreo capaces de registrar continuamente variables como temperatura, velocidad de rotación, torque y desgaste de los equipos, lo que ha permitido avanzar hacia modelos de gestión más inteligentes, sustentados en el análisis de datos históricos y en tiempo real (Hamasha et al., 2025).

A pesar de estos avances, gran parte de las organizaciones continúa utilizando estrategias tradicionales de mantenimiento correctivo y preventivo, las cuales presentan limitaciones importantes: detenciones inesperadas, altos costos de reparación y reemplazos innecesarios de componentes que aún conservan vida útil (Benhanifia et al., 2025). Meitz et al. (2025) señalan que las actividades de mantenimiento pueden representar entre un 15 % y un 70 % de los costos totales de producción, dependiendo del tipo de industria.

El principal desafío técnico consiste en identificar oportunamente las condiciones que anteceden a una falla a partir del gran volumen de datos generado por los sensores, tarea que resulta impracticable de forma manual (Guidotti et al., 2025). En respuesta a esta problemática surge el mantenimiento predictivo, estrategia que utiliza datos históricos y variables de operación para estimar la probabilidad de falla de un equipo y planificar las intervenciones en el momento más oportuno, constituyéndose en una de las principales aplicaciones del aprendizaje automático dentro de la Industria 4.0 (Benhanifia et al., 2025; Hamasha et al., 2025).

La literatura también evidencia desafíos abiertos: Guidotti et al. (2025) destacan la escasez de conjuntos de datos industriales públicos y la necesidad de soluciones más interpretables; Aminzadeh et al. (2025) demuestran, mediante un caso práctico en compresores industriales, que el valor de estas soluciones depende tanto del algoritmo como de su integración en una plataforma funcional. Considerando estos antecedentes, este proyecto continúa el desarrollo de Perfectime, una plataforma de mantenimiento predictivo que integra técnicas de aprendizaje automático supervisado con una interfaz orientada a supervisores de mantenimiento.

2.1. Requisitos iniciales

A partir de la problemática descrita, esta fase mantiene y profundiza los requisitos definidos en la Fase 2 (ver sección 6.b), centrados en tres ejes: (1) ampliar y validar rigurosamente el núcleo predictivo, contrastando múltiples técnicas de Machine Learning y arquitecturas de Deep Learning antes de fijar el modelo de producción; (2) garantizar la trazabilidad de los datos y del entrenamiento (partición, balanceo, métricas) para que los resultados sean reproducibles por terceros; y (3) avanzar en la integración real entre el modelo servido por la API y la interfaz de usuario, reemplazando progresivamente los datos simulados del prototipo visual de la Fase 2.

3. Metodología

El proyecto continúa desarrollándose bajo la metodología CRISP-DM (Cross Industry Standard Process for Data Mining; Chapman et al., 2000), la cual organiza el trabajo en seis etapas: comprensión del negocio, comprensión de los datos, preparación de los datos, modelado, evaluación y despliegue. Las Fases 1 y 2 del proyecto cubrieron íntegramente las cuatro primeras etapas; esta Fase 3 profundiza en modelado y evaluación — ampliando la comparación de técnicas y optimizando hiperparámetros— y avanza en el despliegue, mediante la integración del modelo con la API REST y el frontend.

Las actividades de esta fase se definieron en función de los objetivos y requisitos establecidos para el proyecto. En particular, la comparación de modelos y estrategias de balanceo se relaciona con la selección del modelo más adecuado, mientras que la optimización y evaluación buscan mejorar y validar su desempeño. Por otra parte, el desarrollo del backend y frontend responde a los requisitos asociados a la integración del modelo y la generación de predicciones. Las tareas fueron distribuidas entre los integrantes del equipo y organizadas mediante plazos de trabajo para facilitar el seguimiento del proyecto.

3.1. Fases aplicadas hasta ahora

- **Fase 1 (Comprensión del entorno):** análisis del problema de mantenimiento predictivo, revisión de literatura y definición de objetivos.
- **Fase 2 (Preparación de los datos):** análisis exploratorio, evaluación de calidad, limpieza, codificación y escalado del dataset AI4I 2020, además de un primer modelado con tres técnicas (Random Forest, XGBoost, MLP) y explicabilidad SHAP.
- **Fase 3 (Validación y optimización — este informe):** ampliación a tres técnicas de Machine Learning y tres arquitecturas de Deep Learning, comparación de tres estrategias de balanceo de clases, ajuste de hiperparámetros mediante GridSearchCV, reorganización del repositorio en carpetas estandarizadas (datasets, notebooks, modelos, backend, frontend) y avance en la integración frontend/backend.

3.2. Plan de trabajo detallado

El siguiente plan distribuye las tareas pendientes para el cierre de esta fase y el inicio de la Sumativa 3. Los responsables se mantienen como una propuesta de distribución de carga; el equipo puede reasignarlos según disponibilidad.

Tarea	Responsable(s)	Plazo	Objetivo relacionado
Ampliación del notebook: 3.ª técnica ML (Regresión Logística) y 3 arquitecturas DL (MLP superficial, profunda, ancha)	Claudio Yáñez	11–14 ago 2026	Comparar técnicas y arquitecturas para seleccionar el modelo con mejor desempeño
Reorganización del repositorio (datasets/notebooks/modelos) y corrección de rutas del pipeline	Claudio Yáñez	13–14 ago 2026	Facilitar la ejecución, organización y mantenimiento del proyecto
Redacción del informe Fase 3 (problemática, metodología, resultados, mejoras)	Javiera Chávez, Claudio Yáñez	13–17 ago 2026	Documentar los resultados y justificar las decisiones tomadas durante el desarrollo

Tarea	Responsable(s)	Plazo	Objetivo relacionado
Revisión de bibliografía adicional y formato APA 7	Javiera Chávez	15–17 ago 2026	Respaldar la problemática, metodología y decisiones técnicas mediante fuentes académicas
Conexión de "Carga de archivo CSV" y "Explicabilidad SHAP" del frontend a la API real	Joaquín Olivares	18–24 ago 2026	Avanzar en RF-01 y RF-05 e integrar las funcionalidades del sistema con el modelo
Pruebas de extremo a extremo del sistema (backend + frontend) y ajustes finales	Equipo completo	25–28 ago 2026	Verificar el funcionamiento integrado del sistema y detectar errores antes de la entrega

Tabla 2. Plan de trabajo — Fase 3.

4. Desarrollo del proyecto

4.1. Comparación de técnicas de Machine Learning y arquitecturas de Deep Learning

Para elegir el modelo final se entrenaron y compararon seis modelos —todos entrenados y evaluados de verdad en notebooks/Sumativa.ipynb, sin afirmar nada que no esté respaldado por el código— sobre el mismo conjunto de entrenamiento balanceado con SMOTE (Chawla et al., 2002) y evaluados con el mismo conjunto de prueba (3.000 registros, nunca visto durante el entrenamiento): tres técnicas de Machine Learning —Random Forest (Breiman, 2001), XGBoost (Chen & Guestrin, 2016) y Regresión Logística— y tres arquitecturas de Deep Learning (variantes de Perceptrón Multicapa), todas implementadas con scikit-learn (Pedregosa et al., 2011). Otras técnicas conocidas como SVM (Cortes & Vapnik, 1995) no se incluyeron en esta comparación por tiempo, pero quedan como posible trabajo futuro.

Técnica	Tipo	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Random Forest	ML — Ensemble (Bagging)	0.9627	0.4679	0.7157	0.5659	0.9713
XGBoost	ML — Ensemble (Boosting)	0.9753	0.6077	0.7745	0.6810	0.9780
Regresión Logística	ML — Lineal	0.8353	0.1475	0.8039	0.2492	0.8888
MLP superficial (DL-A)	DL — 1 capa (16 neuronas)	0.9313	0.3169	0.8824	0.4663	0.9692
MLP profunda (DL-B)	DL — 3 capas (64-32-16)	0.9660	0.5000	0.7255	0.5920	0.9492
MLP ancha (DL-C)	DL — 1 capa (128 neuronas)	0.9580	0.4388	0.8431	0.5772	0.9744

Tabla 3. Comparación de las 3 técnicas de ML y las 3 arquitecturas de DL sobre el conjunto de prueba (resultados del notebook, sección 8.2).

En la comparación inicial, XGBoost obtuvo el mejor resultado general (AUC-ROC 0,9780; F1-Score 0,6810), seguido de cerca por dos de las redes neuronales: la MLP ancha (AUC-ROC 0,9744) y la MLP superficial (AUC-ROC 0,9692).

La MLP profunda, que presenta una mayor cantidad de capas, obtuvo un resultado inferior a las otras dos (AUC-ROC 0,9492). Esto muestra que, para este problema y con las variables disponibles, aumentar la profundidad de la red no produjo una mejora en el desempeño.

La Regresión Logística obtuvo el desempeño más bajo (AUC-ROC 0,8888), con un Recall relativamente alto (0,8039), pero una Precision considerablemente menor (0,1475). Esto indica que el modelo logra detectar una proporción importante de los casos de falla, pero a costa de generar una mayor cantidad de falsas alarmas. Por esta razón, para este problema resultaron más adecuados modelos no lineales, como los ensambles de árboles y las redes neuronales.

Para las arquitecturas de Deep Learning se utilizaron tres configuraciones de MLP con el objetivo de observar el efecto de modificar la profundidad y el número de neuronas, manteniendo el mismo problema de clasificación. La MLP superficial utiliza una capa oculta de 16 neuronas, la MLP profunda utiliza tres capas de 64, 32 y 16 neuronas, y la MLP ancha utiliza una capa de 128 neuronas. Esta comparación permitió analizar si aumentar la profundidad o el número de neuronas producía una mejora real en el desempeño. Los resultados muestran que la MLP superficial y la MLP ancha obtuvieron mejores AUC-ROC que la MLP profunda, por lo que aumentar la profundidad no significó una mejora para este conjunto de datos.

4.2. Requisitos funcionales y no funcionales

Se mantienen los requisitos definidos en la Fase 2, actualizados según el estado actual del sistema. Hay un punto que en la fase anterior ya estaba bien hecho en el código, pero no se explicó en el informe: la relación entre el preprocesamiento y el requisito RF-05. Al limpiar el dataset se eliminaron las columnas TWF, HDF, PWF, OSF y RNF porque indican el tipo específico de falla que ocurrió — esa información solo se conoce después de que la falla pasa, así que dejarla en los datos de entrenamiento sería fuga de datos (el modelo estaría "viendo" la respuesta antes de predecirla). Por eso el sistema no puede predecir qué tipo de falla específico va a ocurrir, solo si va a fallar o no. RF-05 se resuelve entonces con el desglose de SHAP (Lundberg & Lee, 2017): en vez de decir "qué tipo de falla", muestra qué variables influyen más, que es información parecida y útil para el supervisor, sin la fuga de datos.

Debido a esta limitación, el alcance actual del modelo corresponde a una clasificación binaria: determinar si existe riesgo de falla o no. La identificación del tipo específico de falla queda fuera del alcance del modelo actual, ya que requeriría plantear un problema de clasificación diferente utilizando información disponible antes de que ocurra la falla. Para mantener la explicabilidad del sistema, se utiliza SHAP para mostrar las variables que tienen mayor influencia en cada predicción, entregando información útil para apoyar el análisis del supervisor.

Requisitos funcionales

ID	Requisito	Estado
RF-01	Carga de archivos CSV por lotes vía POST /predict-batch	Implementado en backend y conectado con frontend
RF-02	Validación de estructura y rangos de los datos mediante esquemas Pydantic	Implementado
RF-03	Preprocesamiento idéntico a entrenamiento (One-Hot + StandardScaler) en la inferencia	Implementado
RF-04	Estimación de probabilidad de falla mediante el modelo XGBoost servido	Implementado y conectado al frontend (predicción individual)

ID	Requisito	Estado
RF-05	Desglose de variables mediante SHAP en cada predicción	Implementado en backend y conectado con frontend
RF-06	Presentación de resultados mediante indicador de riesgo y probabilidad	Implementado (predicción individual)
RF-07	Historial de predicciones con filtros	Implementado
RF-08	Exportación de resultados en CSV	Pendiente
RF-09	Endpoint de monitoreo (GET /health)	Implementado

Tabla 4. Requisitos funcionales y su estado de implementación al cierre de la Fase 3.

Requisitos no funcionales

Categoría	Requisito
Usabilidad	Interfaz intuitiva que permita interpretar fácilmente las predicciones.
Explicabilidad	Mostrar las variables que más influyen en la predicción cuando sea técnicamente posible (SHAP).
Escalabilidad	Arquitectura modular que permita incorporar nuevos conjuntos de datos y modelos sin modificar la estructura principal.
Confiabilidad	Resultados consistentes a partir de datos previamente validados; verificación de integridad del modelo serializado tras cada reentrenamiento.
Monitoreabilidad	Registro de predicciones, archivos procesados y disponibilidad del modelo mediante el endpoint /health.
Mantenibilidad	Estructura de carpetas estandarizada (datasets, notebooks, modelos, backend, frontend) y rutas del pipeline independientes del directorio de ejecución.
Portabilidad	Entorno reproducible mediante environment.yml (Conda) documentado en el README.
Seguridad	Controlar el acceso a las funcionalidades del sistema y validar los datos recibidos antes de procesarlos, evitando entradas que puedan comprometer el funcionamiento del servicio.

Tabla 5. Requisitos no funcionales.

4.3. Arquitectura: justificación de la selección del modelo

PerfectTime usa XGBoost como modelo de producción porque fue el modelo que obtuvo el mejor desempeño general en la comparación inicial (AUC-ROC 0,9780; F1-Score 0,6810), además de permitir manejar el desbalance de clases y ofrecer explicabilidad mediante SHAP (TreeExplainer). Esta elección se apoya en la comparación realizada con las otras cinco técnicas evaluadas, incluyendo las tres arquitecturas de Deep Learning.

La selección de XGBoost también se consideró desde el punto de vista de su posterior integración al sistema. Además de obtener el mejor desempeño inicial entre los modelos comparados, permite utilizar SHAP mediante TreeExplainer para interpretar las predicciones. Posteriormente, se realizó un ajuste de hiperparámetros mediante GridSearchCV, obteniendo una mejora en Precision, Recall y F1-Score respecto del modelo base, mientras que el AUC-ROC presentó una leve disminución. Se mantuvo la configuración optimizada debido a la mejora obtenida en la detección de fallas, especialmente en Recall y F1-Score, manteniendo un modelo adecuado para su integración en el backend.

Arquitectura	Capas ocultas	Neuronas por capa	Activación	Optimizador	AUC-ROC
A — Superficial	1	16	ReLU	Adam	0.9692
B — Profunda	3	64, 32, 16	ReLU	Adam	0.9492
C — Ancha	1	128	ReLU	Adam	0.9744

Tabla 6. Configuración de las tres arquitecturas de Deep Learning comparadas (MLPClassifier, scikit-learn).

Las tres arquitecturas utilizan la misma activación (ReLU) y el mismo optimizador (Adam); lo único que cambia es la profundidad (cantidad de capas) y el ancho (número de neuronas por capa). Así se puede ver el efecto de cada caso por separado. Los resultados (Tabla 3) muestran que la arquitectura ancha (128 neuronas en 1 capa) obtuvo un mejor desempeño que la profunda (3 capas de 64-32-16). Esto es coherente con el hecho de que el dataset solo tiene 7 variables, por lo que aumentar la profundidad de la red no produjo una mejora en este caso.

También se ajustaron los hiperparámetros de XGBoost con GridSearchCV, evaluando 108 combinaciones mediante validación cruzada de 5 folds, para un total de 540 ajustes. La mejor configuración fue `n_estimators=300`, `max_depth=7`, `learning_rate=0,2`, `subsample=0,8` y `colsample_bytree=1,0`. En el conjunto de prueba, el modelo optimizado obtuvo un Accuracy de 0,9773, Precision de 0,6328, Recall de 0,7941, F1-Score de 0,7043 y AUC-ROC de 0,9769. En comparación con el modelo base, se observa una mejora en Precision, Recall y F1-Score, mientras que el AUC-ROC disminuye levemente.

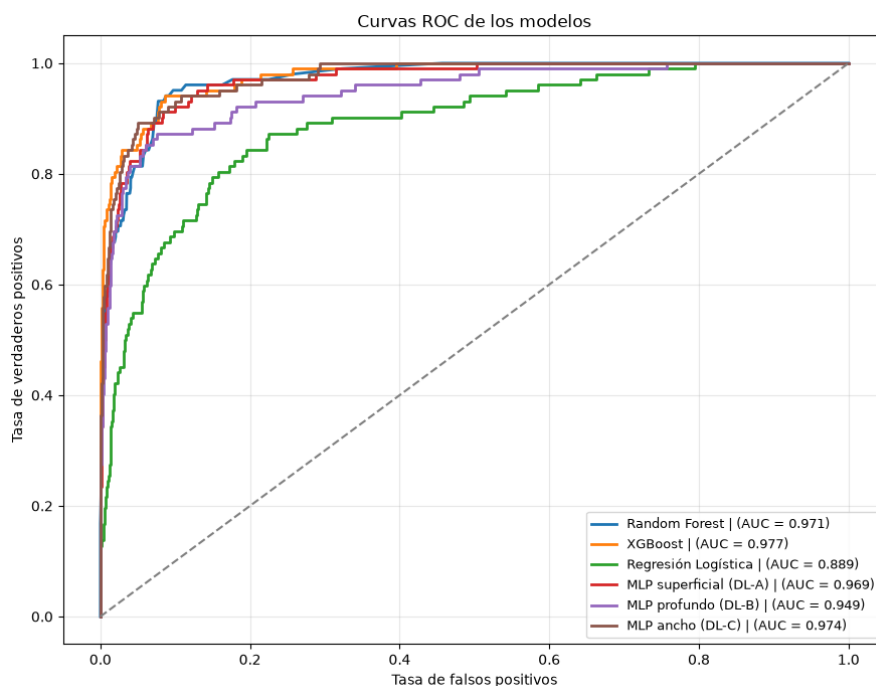


Figura 1. Curvas ROC de las 6 técnicas comparadas.

4.4. Modelos: partición de datos y balanceo de clases

El dataset AI4I 2020 (Matzka, 2020) (10.000 registros, 7 variables predictoras) se dividió en 70 % entrenamiento (7.000 registros) y 30 % prueba (3.000 registros), usando muestreo estratificado (stratify) para que ambas partes mantengan la misma proporción de fallas (3,39 %).

Como hay un desbalance grande entre las clases —algo común en este tipo de problemas (He & Garcia, 2009)—, se compararon tres formas de tratarlo sobre el conjunto de entrenamiento, dejando fijo el conjunto de prueba, el modelo (XGBoost) y la semilla aleatoria:

Escenario	N° entrenamiento	% clase positiva	Accuracy	Precision	Recall	F1-Score	AUC-ROC
1. Sin balanceo	7.000	3.39 %	0.9833	0.8250	0.6471	0.7253	0.9793
2. Submuestreo (Undersampling)	474	50.00 %	0.9037	0.2533	0.9412	0.3992	0.9655
3. SMOTE (Oversampling sintético)	13.526	50.00 %	0.9753	0.6077	0.7745	0.6810	0.9780

Tabla 7. Comparación de tres técnicas de balanceo de clases (XGBoost, notebook sección 8.3).

Sin balanceo se obtuvo la mejor Precision y Accuracy, pero el menor Recall (0,6471), lo que significa que se detectó una menor proporción de las fallas reales. En un problema de mantenimiento predictivo, este aspecto es especialmente relevante, ya que no detectar una falla puede tener consecuencias operativas. El undersampling aumentó el Recall hasta 0,9412, pero produjo una fuerte disminución de Precision y F1-Score. SMOTE presentó un equilibrio más favorable entre las métricas, con un F1-Score de 0,6810 y un AUC-ROC de 0,9780. Por esta razón, se seleccionó SMOTE como estrategia de balanceo para las etapas posteriores.

Esta comparación permitió seleccionar la estrategia de balanceo a partir de los resultados obtenidos y no solo por elección teórica. En este caso, el Recall es especialmente importante porque representa la capacidad de detectar máquinas que realmente presentan una falla. Aunque el escenario sin balanceo obtuvo mayor Accuracy, se prefirió SMOTE por ofrecer un mejor equilibrio entre Precision, Recall y F1-Score, manteniendo además un AUC-ROC elevado.

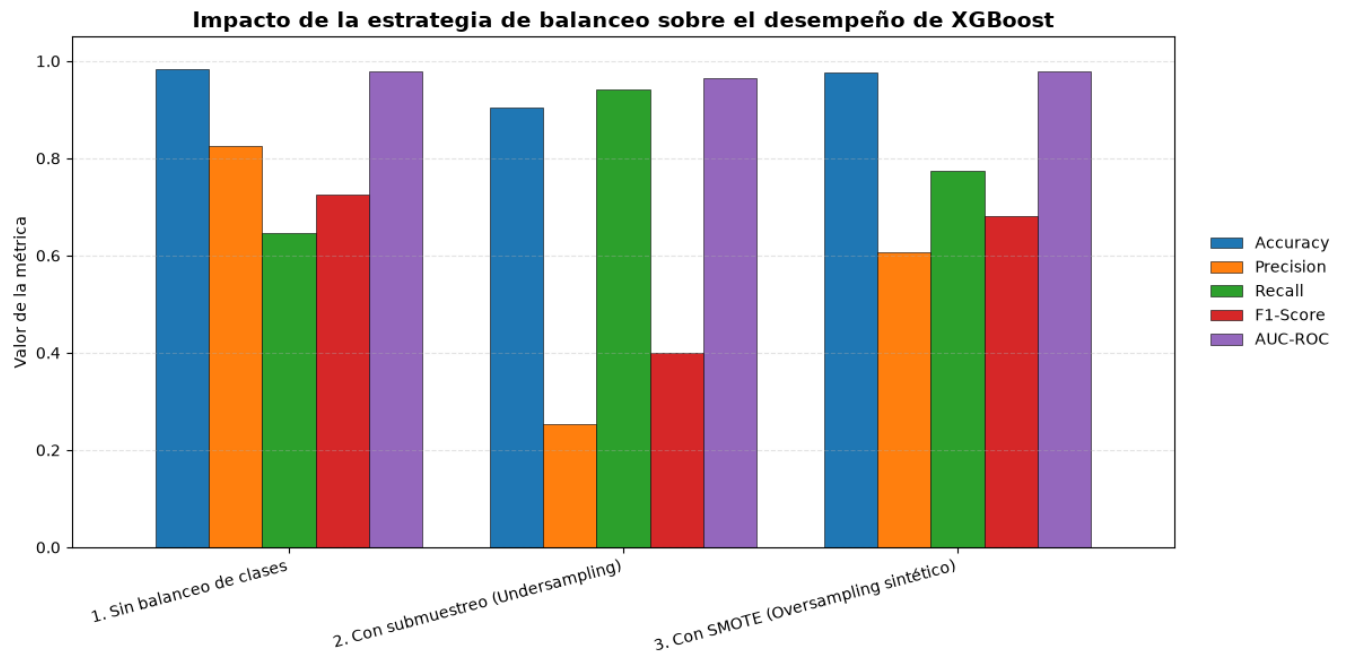


Figura 2. Impacto de la estrategia de balanceo sobre el desempeño de XGBoost.

4.5. Sistema: avance en la integración frontend/backend

En esta etapa se avanzó desde una maqueta visual hacia una integración funcional entre frontend y backend. Se construyó el servicio FastAPI (backend/main.py), que implementa la lógica de predicción, validación y procesamiento de datos, utilizando el modelo XGBoost, el StandardScaler y el orden de columnas definido durante el entrenamiento. El backend dispone de endpoints para predicción individual, predicción por lotes, consulta del historial, reentrenamiento y monitoreo del sistema.

La pantalla de “Predicción individual” del frontend (frontend/index.html) se encuentra conectada al backend. El formulario solicita los cinco valores reales de los sensores y el tipo de producto, y envía los datos al endpoint /predict, que realiza el preprocesamiento correspondiente y genera la predicción mediante el modelo XGBoost. Además, se implementó la carga de archivos CSV mediante el endpoint /predict-batch, permitiendo procesar múltiples registros. Las funcionalidades de “Historial”, “Explicabilidad SHAP” y “Monitoreo” también se encuentran conectadas con sus respectivos endpoints del backend y muestran información generada a partir de las predicciones realizadas por el sistema.

Como evidencia, se muestra a continuación una petición realizada directamente al backend y la respuesta obtenida para un caso de prueba. La solicitud utiliza datos de sensores y el servicio devuelve la predicción, la probabilidad y el nivel de riesgo correspondiente.

```
POST /predict
{"air_temperature_k":303.5,"process_temperature_k":312.0,"rotational_speed_rpm":1350,"torque_nm":68.5,"tool_wear_min":220,"type":"L"}
→ 200 OK
{"prediccion":1,"etiqueta":"Falla","probabilidad_porcentaje":100.0,"nivel_riesgo":"Falla","factor_principal":"Torque"}
```

Con valores normales (ej. 298,1 K / 1.551 rpm / 42,8 Nm / 108 min) el mismo endpoint responde "No falla" con 0 % de probabilidad. Esto muestra que el modelo distingue bien entre los dos casos y no devuelve siempre lo mismo: el usuario ingresa datos reales y recibe una predicción calculada por el modelo, no un valor inventado.

Este avance permite validar la integración básica entre los principales componentes de PerfectTime: el frontend recibe los datos ingresados por el usuario, el backend realiza el preprocesamiento y utiliza el modelo XGBoost para generar la predicción, y las distintas funcionalidades del sistema consumen los resultados obtenidos. La integración fue comprobada mediante predicciones individuales, carga de archivos CSV, consulta del historial, monitoreo y visualización de explicaciones SHAP. Como funcionalidad pendiente se mantiene la exportación de resultados en CSV, mientras que las demás funcionalidades principales se encuentran operativas.

5. Resultados

El modelo seleccionado (XGBoost, optimizado mediante GridSearchCV) fue evaluado sobre el conjunto de prueba de 3.000 registros, no utilizado durante el entrenamiento ni el balanceo de clases.

Métrica	XGBoost (línea base)	XGBoost (optimizado)
Accuracy	0.9753	0.9773
Precision (clase Con falla)	0.6077	0.6328
Recall (clase Con falla)	0.7745	0.7941
F1-Score (clase Con falla)	0.6810	0.7043

Métrica	XGBoost (línea base)	XGBoost (optimizado)
AUC-ROC	0.9780	0.9769

Tabla 8. Métricas del modelo final sobre datos no vistos.

Los resultados muestran que el ajuste de hiperparámetros produjo una mejora moderada respecto del modelo base. La exactitud aumentó de 0.9753 a 0.9773, mientras que la precisión pasó de 0.6077 a 0.6328, el recall de 0.7745 a 0.7941 y el F1-Score de 0.6810 a 0.7043. Por otra parte, el AUC-ROC presentó una leve disminución, pasando de 0.9780 a 0.9769. En conjunto, la optimización permitió mejorar principalmente la capacidad del modelo para identificar los casos de falla, manteniendo un desempeño general elevado. Por esta razón, el modelo XGBoost optimizado continúa siendo la alternativa seleccionada para el sistema.

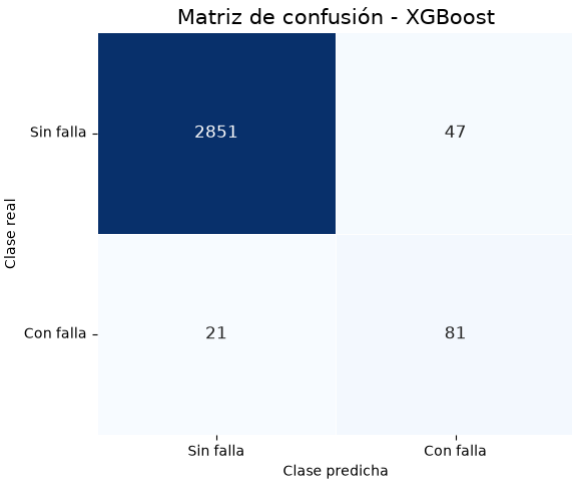


Figura 3. Matriz de confusión — XGBoost.

Desde el punto de vista del mantenimiento predictivo, los 21 falsos negativos son especialmente relevantes, ya que corresponden a máquinas que presentaban una falla pero fueron clasificadas como si no la tuvieran. Este tipo de error puede ser más crítico que un falso positivo, porque podría provocar que una máquina continúe operando sin una revisión preventiva. Por otra parte, los 47 falsos positivos representan casos en los que el modelo indicó una posible falla cuando esta no estaba presente, lo que puede generar revisiones o intervenciones innecesarias. Por lo tanto, los resultados muestran que el modelo presenta un buen desempeño general, aunque existe margen para reducir ambos tipos de error, especialmente los falsos negativos.

5.1. Análisis interpretativo mediante SHAP

Se aplicó SHAP (*SHapley Additive exPlanations*; Lundberg & Lee, 2017) sobre el modelo XGBoost, usando TreeExplainer, para ver qué variables influyen más en las predicciones (Molnar, 2022). El gráfico de importancia global (SHAP Summary Plot) muestra que Tool_wear_min (desgaste de la herramienta), Torque_Nm y Air_temperature_K son las variables con mayor peso en las predicciones. A continuación, se encuentran Rotational_speed_rpm y Process_temperature_K, mientras que el tipo de producto (Type_L y Type_M) presenta la menor influencia.

Estos resultados permiten complementar las métricas de evaluación, ya que no solo muestran qué tan bien funciona el modelo, sino también qué variables tienen mayor influencia en sus predicciones. En este caso, el desgaste de la herramienta y el torque aparecen como los factores de mayor influencia, mientras que las variables asociadas al tipo de producto presentan un aporte menor. Esto permite interpretar la contribución de las variables al resultado del modelo, pero no implica que exista una relación causal entre ellas y la ocurrencia de una falla.

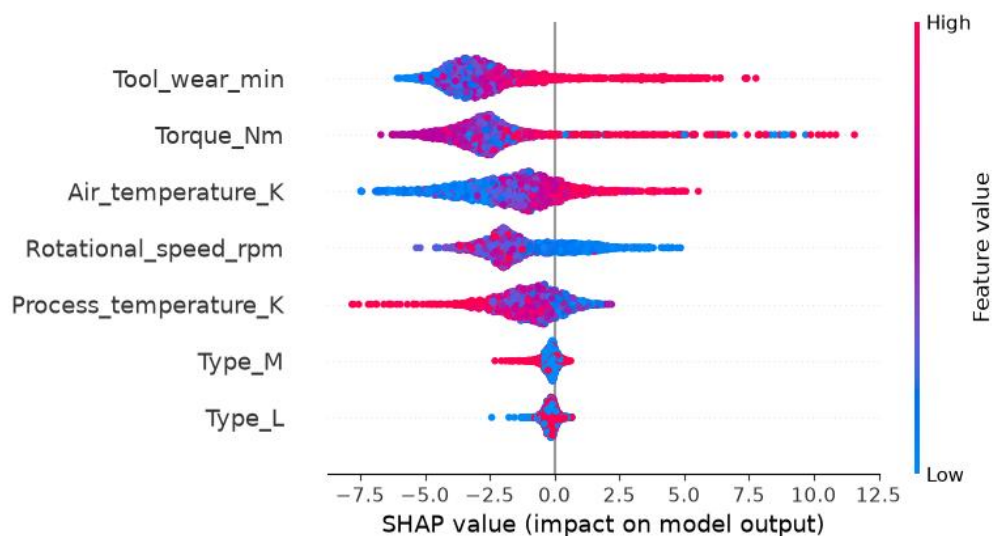


Figura 4. SHAP Summary Plot — importancia global de las variables (modelo XGBoost).

6. Propuesta de mejoras

6.1. Limitaciones identificadas

- **Naturaleza estática de los datos tabulares:** el dataset AI4I 2020 entrega mediciones en instantes discretos, lo que limita la capacidad de los modelos actuales para capturar la evolución temporal de la degradación de un componente previa a una falla por fatiga o desgaste.
- **Sensibilidad al desbalanceo severo en tipos de falla infrecuentes:** aunque SMOTE mejora el balance agregado de la clase "con falla", tipos de falla específicos y extremadamente infrecuentes dentro de esa clase (p. ej., fallas aleatorias) quedan subrepresentados incluso tras el remuestreo, aumentando el riesgo de falsos negativos en escenarios críticos.

El modelo actual fue planteado como una clasificación binaria, por lo que no permite identificar directamente cuál de los cinco tipos de falla ocurrió. Esta decisión se tomó para evitar utilizar como variables predictoras TWF, HDF, PWF, OSF y RNF, ya que corresponden a indicadores asociados a la falla y su utilización podría generar fuga de información. Una mejora futura sería abordar la identificación del tipo de falla como un problema de clasificación diferente, utilizando únicamente información disponible antes de que ocurra el evento.

6.2. Mejoras propuestas

Las mejoras propuestas se enfocan en las principales limitaciones identificadas durante la evaluación. La primera busca incorporar información temporal y capturar mejor la evolución de las variables antes de una falla, mientras que la segunda busca mejorar la adaptación del sistema frente a cambios en los datos y reducir errores mediante retroalimentación. La primera corresponde a una extensión futura. La segunda -el reentrenamiento del modelo con datos nuevos- ya cuenta con una primera implementación funcional (endpoint POST/reentrenar, accesible desde la pantalla de Monitoreo): el supervisor puede cargar registros históricos con el resultado real de la falla, y el sistema reentrena y reemplaza el modelo en producción sin reiniciar la API. Lo que queda como trabajo futuro es automatizar la detección de predicciones inciertas (aprendizaje activo) y el monitoreo de deriva de datos (data drift), en vez de depender de una carga manual.

- **Mejora 1 — Ventanas temporales deslizantes y arquitectura híbrida CNN-LSTM:** transformar la ingesta de telemetría en ventanas secuenciales (agrupando los últimos minutos de operación) y entrenar una arquitectura híbrida 1D-CNN + LSTM sobre esa estructura. La capa convolucional extraería patrones de

fluctuación local en torque y temperatura, mientras que la celda LSTM modelaría la acumulación paulatina de estrés mecánico, permitiendo estimar no solo si ocurrirá una falla, sino la Vida Útil Restante (RUL) del equipo. Esta mejora responde directamente a la limitación de datos estáticos identificada arriba.

- **Mejora 2 — Aprendizaje activo y monitoreo de deriva de datos (data drift):** integrar un módulo que identifique automáticamente las predicciones con menor margen de certidumbre (p. ej., probabilidades entre 0,45 y 0,55) y las envíe a la interfaz del supervisor para validación humana, retroalimentando y reentrenando periódicamente el modelo. En entornos industriales reales las condiciones operativas cambian constantemente (concept drift), degradando la precisión del modelo con el tiempo; esta mejora reduce los falsos positivos/negativos y asegura la adaptabilidad continua del sistema.

7. Código fuente en GitHub

El código fuente completo del proyecto —incluyendo el notebook de análisis y entrenamiento, backend, frontend, dataset, modelos serializados y archivos necesarios para la ejecución— se encuentra disponible públicamente en el siguiente repositorio:

[Repositorio GitHub — Perfectime](#)

Durante esta fase, el repositorio fue reorganizado en carpetas estandarizadas (datasets/, notebooks/, modelos/, backend/, frontend/ y Documentacion/). Además, se corrigieron las rutas utilizadas por el pipeline para que la lectura del dataset y el almacenamiento de los modelos no dependan del directorio desde el cual se ejecute Jupyter.

El archivo README.md contiene las instrucciones necesarias para crear el entorno Conda, instalar las dependencias y ejecutar el notebook, el backend y el frontend. De esta forma, el repositorio permite reproducir el flujo completo del proyecto y facilita la revisión del código desarrollado.

Video de demostración: [enlace a completar por el equipo antes de la entrega] — video corto mostrando el flujo de predicción individual en funcionamiento, según lo sugerido por el docente.

8. Referencias

- Aminzadeh, A., Sattarpanah Karganroudi, S., Majidi, S., Dabompre, C., Azaiez, K., Mitride, C., & Sénéchal, E. (2025). A machine learning implementation to predictive maintenance and monitoring of industrial compressors. *Sensors*, 25(4), 1006. <https://doi.org/10.3390/s25041006>
- Benhanifia, A., Ben Cheikh, Z., Oliveira, P. M., Valente, A., & Lima, J. (2025). Systematic review of predictive maintenance practices in the manufacturing sector. *Intelligent Systems with Applications*, 26, 200501. <https://doi.org/10.1016/j.iswa.2025.200501>
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc.
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357. <https://doi.org/10.1613/jair.953>

- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. En *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM. <https://doi.org/10.1145/2939672.2939785>
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297. <https://doi.org/10.1007/BF00994018>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- Guidotti, D., Pandolfo, L., & Pulina, L. (2025). A systematic literature review of supervised machine learning techniques for predictive maintenance in Industry 4.0. *IEEE Access*, 13, 102479–102504. <https://doi.org/10.1109/ACCESS.2025.3578686>
- Hamasha, M. M., Albedoor, Q., Hamasha, S., Ali, H., Qamar, A., & Berrah, F. (2025). A comprehensive framework for IoT-driven predictive maintenance: Leveraging AI and edge computing for enhanced equipment reliability. *Journal of Applied Engineering Science*, 23(3), 471–486. <https://doi.org/10.5937/jaes0-57002>
- He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284. <https://doi.org/10.1109/TKDE.2008.239>
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. En I. Guyon, U. von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, & R. Garnett (Eds.), *Advances in Neural Information Processing Systems 30* (pp. 4765–4774). Curran Associates.
- Matzka, S. (2020). Explainable artificial intelligence for predictive maintenance applications. En *2020 Third International Conference on Artificial Intelligence for Industries (AI4I)* (pp. 69–74). IEEE. <https://doi.org/10.1109/AI4I49448.2020.00023>
- Meitz, L., Senge, J., Wagenhals, T., Schöler, T., Hähner, J., Edinger, J., & Krupitzer, C. (2025). A literature review framework and open research challenges for predictive maintenance in Industry 4.0. *Computers & Industrial Engineering*, 206, 111193. <https://doi.org/10.1016/j.cie.2025.111193>
- Molnar, C. (2022). *Interpretable machine learning: A guide for making black box models explainable* (2nd ed.). <https://christophm.github.io/interpretable-ml-book/>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.